LAB IMPLEMENTATION EXPERIMENT 11

Experiment 11: Style Transfer Using Generative AI

Generative AI Laboratory (CO2, CO3)

Date: 22 October 2026

Aim

To implement neural style transfer with a pre-trained convolutional network, creating images that combine the content of a photograph with the artistic style of a painting.

Theory

Neural style transfer (Gatys, Ecker and Bethge, 2015) treats a pre-trained image-classification CNN (VGG19) as a feature extractor. Deep layers respond to *what* is in the image (objects and layout), whereas the *correlations* between feature channels in several layers describe the *texture and style*. For a layer with feature map $F \in \mathbb{R}^{N \times C}$ the **Gram matrix** is $G = F^{\top} F / N$.

A new image x is obtained by gradient descent **on the pixels** (the network weights stay frozen) to minimise

$$\mathcal{L}(x) = \alpha \, \| F^{l}(x) - F^{l}(c) \|^2 + \beta \sum_{l} w_l \, \| G^{l}(x) - G^{l}(s) \|^2 + \gamma \, TV(x),$$

where c is the content image, s the style image, and the total-variation term TV suppresses noise. The ratio β/α controls how strongly the style is imposed. The method is *generative* because it synthesises a new image, but it needs an optimisation for every image pair; later feed-forward models (fast style transfer, AdaIN) learn to stylise in one pass.

Steps

1. Download two content photographs and one painting (Kandinsky, *Composition 7*) and resize them to 256 pixels.
2. Load VGG19 with ImageNet weights (without the classifier) and select `block5_conv2` for content and five `conv1` layers for style.
3. Implement Gram matrices and the content, style and total-variation losses.
4. Optimise the pixels of the image with Adam for 250 steps; log the losses and save intermediate images.
5. Repeat with a second content image and with three different style weights.
6. Quantify the style distance to the painting and the content distance to the photo.

Code

```
import os, time
os.environ["TF_CPP_MIN_LOG_LEVEL"] = "3"
```

```
import numpy as np, matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from PIL import Image
keras.utils.set_random_seed(11)
MAX_DIM = 256
print("TensorFlow", tf.__version__, "| image size (longest side):", MAX_DIM)

base = "https://storage.googleapis.com/download.tensorflow.org/example_images/"
files = {"labrador": "YellowLabradorLooking_new.jpg", "turtle": "
    Green_Sea_Turtle_grazing_seagrass.jpg",
         "kandinsky": "Vassily_Kandinsky%2C_1913_-_Composition_7.jpg"}
paths = {k: keras.utils.get_file(v.replace("%2C", ","), base + v) for k, v in files.
    items()}
def load_img(path):
    img = tf.io.decode_image(tf.io.read_file(path), channels=3, dtype=tf.float32)
    scale = MAX_DIM / max(img.shape[:2])
    new = tf.cast(tf.round(tf.cast(img.shape[:2], tf.float32) * scale), tf.int32)
    return tf.image.resize(img, new)[tf.newaxis]
imgs = {k: load_img(p) for k, p in paths.items()}
for k, v in imgs.items(): print(f"{k:10s} -> tensor {tuple(v.shape)}, range [{v.
    numpy().min():.2f}, {v.numpy().max():.2f}]")
fig, axes = plt.subplots(1, 3, figsize=(12, 3.6))
for a, (k, t) in zip(axes, [("Content 1: labrador", "labrador"), ("Content 2:
    turtle", "turtle"), ("Style: Kandinsky, Composition 7", "kandinsky")]):
    a.imshow(imgs[t][0]); a.set_title(k); a.axis("off")
plt.tight_layout(); plt.show()

content_layers = ["block5_conv2"]
style_layers = ["block1_conv1", "block2_conv1", "block3_conv1", "block4_conv1", "
    block5_conv1"]
vgg = keras.applications.VGG19(include_top=False, weights="imagenet")
vgg.trainable = False
extractor = keras.Model(vgg.input, [vgg.get_layer(n).output for n in style_layers +
     content_layers])
print("VGG19 layers used for style  :", style_layers)
print("VGG19 layer used for content :", content_layers)
print("VGG19 parameters (frozen)    : {:,}".format(vgg.count_params()))

def features(x):
    out = extractor(keras.applications.vgg19.preprocess_input(x * 255.0))
    style = [tf.linalg.einsum("bijc,bijd->bcd", f, f) / tf.cast(tf.shape(f)[1] * tf.
    shape(f)[2], tf.float32) for f in out[:len(style_layers)]]
    return style, out[len(style_layers):]

def stylize(content, style, steps=250, style_w=1e-2, content_w=1e4, tv_w=30, lr
    =0.02, snap=(0, 50, 125)):
    style_t, _ = features(style); _, content_t = features(content)
    img = tf.Variable(content)
    opt = keras.optimizers.Adam(lr, beta_1=0.99, epsilon=1e-1)
    @tf.function
    def step():
        with tf.GradientTape() as tape:
            s, c = features(img)
            ls = tf.add_n([tf.reduce_mean((a - b) ** 2) for a, b in zip(s, style_t)
    ]) / len(style_layers)
            lc = tf.add_n([tf.reduce_mean((a - b) ** 2) for a, b in zip(c,
    content_t)])
```

```
            tv = tf.reduce_sum(tf.image.total_variation(img))
            loss = style_w * ls + content_w * lc + tv_w * tv
        opt.apply_gradients([(tape.gradient(loss, img), img)]); img.assign(tf.clip_by_value(img, 0, 1))
        return loss, ls, lc
    snaps, log = {}, []
    for i in range(steps + 1):
        if i in snap: snaps[i] = img.numpy()[0].copy()
        loss, ls, lc = step()
        if i % 50 == 0: log.append((i, float(loss), float(ls), float(lc)))
    snaps[steps] = img.numpy()[0]
    return snaps, log
print("stylize() defined")

t0 = time.time()
snaps1, log1 = stylize(imgs["labrador"], imgs["kandinsky"], steps=250)
print(f"{'step':>5s} {'total loss':>12s} {'style loss':>12s} {'content loss':>14s}")
for r in log1: print(f"{r[0]:5d} {r[1]:12.4f} {r[2]:12.2f} {r[3]:14.2f}")
print(f"elapsed {time.time() - t0:.0f}s")

fig, axes = plt.subplots(1, 4, figsize=(14, 3.6))
axes[0].imshow(imgs["labrador"][0]); axes[0].set_title("content")
for a, k in zip(axes[1:], sorted(snaps1)): a.imshow(snaps1[k]); a.set_title(f"after {k} steps")
for a in axes: a.axis("off")
plt.tight_layout(); plt.show()

snaps2, _ = stylize(imgs["turtle"], imgs["kandinsky"], steps=250)
snaps_lo, _ = stylize(imgs["labrador"], imgs["kandinsky"], steps=250, style_w=1e-3)
snaps_hi, _ = stylize(imgs["labrador"], imgs["kandinsky"], steps=250, style_w=1e-1)
fig, axes = plt.subplots(2, 3, figsize=(12, 7))
axes[0, 0].imshow(imgs["turtle"][0]); axes[0, 0].set_title("content: turtle")
axes[0, 1].imshow(imgs["kandinsky"][0]); axes[0, 1].set_title("style")
axes[0, 2].imshow(snaps2[250]); axes[0, 2].set_title("turtle + Kandinsky")
axes[1, 0].imshow(snaps_lo[250]); axes[1, 0].set_title("style weight 1e-3 (weak style)")
axes[1, 1].imshow(snaps1[250]); axes[1, 1].set_title("style weight 1e-2")
axes[1, 2].imshow(snaps_hi[250]); axes[1, 2].set_title("style weight 1e-1 (strong style)")
for a in axes.ravel(): a.axis("off")
plt.tight_layout(); plt.show()

def style_dist(img, ref):
    s, _ = features(img[None]); sr, _ = features(ref)
    return float(tf.add_n([tf.reduce_mean((a - b) ** 2) for a, b in zip(s, sr)]) / len(style_layers))
def content_dist(img, ref):
    _, c = features(img[None]); _, cr = features(ref)
    return float(tf.add_n([tf.reduce_mean((a - b) ** 2) for a, b in zip(c, cr)]))
rows = [("original photo", imgs["labrador"][0].numpy()), ("style weight 1e-3", snaps_lo[250]),
        ("style weight 1e-2", snaps1[250]), ("style weight 1e-1", snaps_hi[250])]
print(f"{'image':20s} {'style distance to painting':>28s} {'content distance to photo':>28s}")
for n, im in rows:
    print(f"{n:20s} {style_dist(im, imgs['kandinsky']):28.2f} {content_dist(im, imgs['labrador']):28.2f}")
```

Listing 1. `experiment_code.py`

Output

Imports and configuration

```
TensorFlow 2.21.0 | image size (longest side): 256
```

Downloading and displaying the images

```
labrador   -> tensor (1, 211, 256, 3), range [0.00, 1.00]
turtle     -> tensor (1, 192, 256, 3), range [0.00, 1.00]
kandinsky  -> tensor (1, 168, 256, 3), range [0.00, 1.00]
```



Building the VGG19 feature extractor

```
VGG19 layers used for style  : ['block1_conv1', 'block2_conv1', 'block3_conv1', 'block4_conv1', '
    block5_conv1']
VGG19 layer used for content : ['block5_conv2']
VGG19 parameters (frozen)    : 20,024,384
```

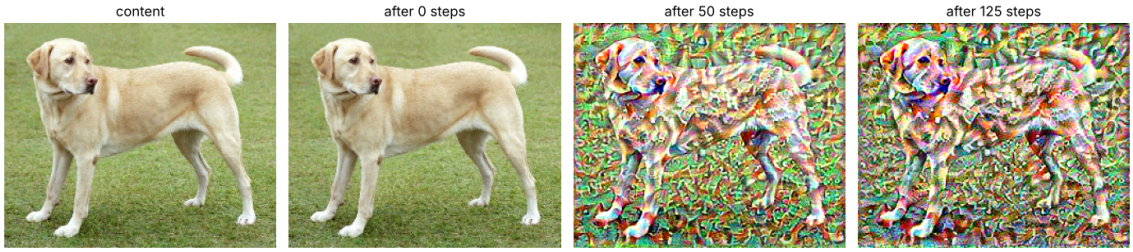
Style-transfer optimisation routine

```
stylize() defined
```

Labrador in the style of Kandinsky - optimisation log

```
 step   total loss   style loss   content loss
    0 2836684544.0000 283620507648.00           0.00
   50 124583736.0000 8091668480.00        4220.64
  100 77349568.0000 4214853376.00        3359.87
  150 54955580.0000 2662961152.00        2671.22
  200 41960824.0000 1859768192.00        2177.62
  250 34034020.0000 1383740544.00        1864.68
elapsed 116s
```

Evolution of the stylised image



Second content image (turtle) and effect of the style weight



Quantifying content preservation and style transfer

image	style distance to painting	content distance to photo
original photo	283620474880.00	0.00
style weight 1e-3	2755334656.00	473.16
style weight 1e-2	1376410368.00	1858.36
style weight 1e-1	1006171968.00	6207.12

Conclusion

Neural style transfer was implemented with a frozen VGG19. In 250 optimisation steps (about 2 minutes per image on a CPU) the style loss fell from 2.8×10^{11} to 1.4×10^{9} while the content loss stayed at 1865, so the Labrador and the turtle kept their shapes but acquired Kandinsky's colours, strokes and textures. The style weight controls the balance: raising it from 10^{-3} to 10^{-1} lowered the style distance to the painting from 2.76×10^{9} to 1.01×10^{9} and raised the content distance from 473 to 6207, i.e. a stronger style costs a less recognisable photograph. The method needs an optimisation for every image pair, which motivates the feed-forward style-transfer networks used

in practice.